FlairSim

Simulateur de drone sur imagerie aérienne FLAIR-HUB

Guide complet d'installation, d'architecture et d'utilisation

Fil rouge — Mastère Spécialisé Big Data / Intelligence Artificielle
Télécom Paris

21 février 2026

Inspiré de FLAIR-HUB [1] et FlySearch [2]

Table des matières

1 Introduction **3**
1.1 Objectifs . . . 3

2 Installation **3**
2.1 Prérequis . . . 3
2.2 Installation . . . 3
2.3 Groupes de dépendances . . . 4
2.4 Fichiers gérés par uv . . . 4
2.5 Note macOS — bug UF_HIDDEN . . . 4

3 Architecture **4**
3.1 map/ — Gestion géospatiale des tuiles . . . 5
3.2 drone/ — État du drone et capteurs . . . 5
3.3 Modèle de caméra . . . 6
3.4 core/ — Moteur de simulation . . . 6
3.5 server/ — API HTTP REST et SSE . . . 6
3.5.1 Server-Sent Events (SSE) . . . 7
3.6 viewer/ — Visualisation desktop . . . 7

4 Données FLAIR-HUB **8**
4.1 Structure des données . . . 8
4.2 Modalités supportées . . . 8
4.3 Données disponibles localement . . . 8

5 Utilisation **8**
5.1 Lancer le serveur . . . 8
5.2 Piloter le drone depuis un programme externe . . . 9
5.3 Viewer interactif . . . 10
5.3.1 Mode local (défaut) . . . 10
5.3.2 Mode observe . . . 10
5.3.3 Mode fly . . . 10
5.3.4 Contrôles clavier . . . 11
5.4 Format de la réponse d'observation . . . 11
5.5 Types d'action . . . 12

6 Choix de conception **12**
6.1 Pourquoi pas un moteur 3D (UE5, Unity) ? . . . 12
6.2 Pourquoi des déplacements relatifs ? . . . 12
6.3 Pourquoi une API HTTP ? . . . 12
6.4 Pourquoi des dataclasses gelés ? . . . 12

7 Flux de données **13**

8 Tests **13**

9 Structure du projet **14**

10 Utilisation programmatique (in-process) **15**

11 Qualité du code **15**
11.1 Linting et formatage . . . 15
11.2 Conventions . . . 15

12 Références **16**

1 Introduction

FlairSim est un simulateur de drone léger, écrit entièrement en Python, conçu pour survoler de l'imagerie aérienne réelle issue du jeu de données **FLAIR-HUB** [1] publié par l'IGN (Institut national de l'information géographique et forestière).

Le projet s'inscrit dans le cadre du fil rouge du Mastère Spécialisé Big Data / IA de Télécom Paris. Il s'inspire de deux travaux de recherche :

1. **FLAIR-HUB** (arXiv :2506.07080) : jeu de données multimodal à grande échelle pour la cartographie de l'occupation des sols en France.
2. **FlySearch** (arXiv :2506.02896, NeurIPS 2025) : benchmark évaluant les capacités d'exploration de modèles vision-langage (VLM) avec un drone.

FlairSim expose une **API HTTP REST locale** permettant à tout programme externe (agent VLM, notebook, script dans un autre langage) de piloter le drone et de recevoir des observations (image + télémétrie). Le simulateur ne contient aucune implémentation d'agent : c'est un *pur package de simulation*.

1.1 Objectifs

— Voler au-dessus d'images aériennes **réelles** (0.2 m/px) plutôt que d'environnements 3D synthétiques.
— Fournir une API simple et documentée pour le pilotage par un agent externe.
— Permettre l'observation en temps réel via Server-Sent Events (SSE).
— Garantir la reproductibilité : simulation déterministe, pas de moteur physique aléatoire.
— Rester léger : pure Python, pas de dépendance à un moteur de jeu.

2 Installation

2.1 Prérequis

— `uv` ≥ 0.4 (gestionnaire de paquets et d'environnements Python).
— Données FLAIR-HUB : au moins un répertoire `D0xx_AERIAL_RGBI` contenant des ROI avec des fichiers GeoTIFF.

`uv` gère automatiquement la version de Python (3.11, épinglée dans `.python-version`) et l'environnement virtuel (`.venv/`).

2.2 Installation

```
cd simulateur

# Installer tout (core + server + viewer + dev)
uv sync --all-extras
```

2.3 Groupes de dépendances

Groupe	Commande	Paquets
Core	`uv sync`	numpy, rasterio, Pillow
Server	`uv sync -extra server`	+ fastapi, uvicorn, sse-starlette
Viewer	`uv sync -extra viewer`	+ pygame
Dev	`uv sync -dev`	+ pytest, pytest-asyncio, ruff, httpx
Tout	`uv sync -all-extras`	Toutes les dépendances ci-dessus

Table 1 – Groupes d'installation

2.4 Fichiers gérés par uv

Fichier	Rôle	Git
`pyproject.toml`	Dépendances et configuration du projet	oui
`uv.lock`	Versions résolues (reproductibilité)	oui
`.python-version`	Version Python épinglée (3.11)	oui
`.venv/`	Environnement virtuel local	non

Table 2 – Fichiers du système de build

2.5 Note macOS — bug UF_HIDDEN

Avec `uv sync` en mode éditable, macOS peut positionner le flag `UF_HIDDEN` sur les fichiers `.pth` en raison de l'attribut étendu `com.apple.provenance` ajouté par le Python installé via `uv`. Le module `site.py` de Python 3.11 ignore les fichiers `.pth` cachés, ce qui casse les console scripts.

Solution : utiliser `uv run python -m flairsim.server` (et non `flairsim-server`) pour lancer le serveur. Les installations non-éditables (`pip install .`) ne sont pas affectées.

3 Architecture

Le package `flairsim` est organisé en cinq sous-packages, chacun avec une responsabilité bien définie :

```
flairsim/
  map/        Gestion geospatiale des tuiles       (couche donnees)
  drone/      Etat du drone, physique, capteurs     (couche agent)
  core/       Boucle de simulation, actions, obs    (couche moteur)
  server/     API HTTP REST + SSE (FastAPI)         (couche interface)
  viewer/     Visualisation temps reel (pygame)     (couche
     presentation)
```

Les dépendances coulent strictement vers le bas :

```
      server
        |
      core         viewer
     /    \          |
  drone   map       core
    \    /
    (numpy)
```

Aucune dépendance circulaire n'existe.

3.1 map/ — Gestion géospatiale des tuiles

Ce module charge les fichiers GeoTIFF FLAIR-HUB depuis le disque, les assemble en une surface raster continue, et fournit des sous-régions rectangulaires arbitraires au modèle de caméra.

Classe	Fichier	Rôle
`Modality`	`modality.py`	Enum des 10 modalités FLAIR-HUB
`ModalitySpec`	`modality.py`	Dataclass gelé décrivant les propriétés physiques
`TileInfo`	`tile_loader.py`	Métadonnées légères d'une tuile
`TileData`	`tile_loader.py`	Tuile chargée en mémoire
`MapBounds`	`map_manager.py`	Boîte englobante avec intersection/inclusion
`MapManager`	`map_manager.py`	Moteur spatial : découverte, géométrie, extraction

Table 3 – Classes du module `map/`

Décisions clés :

- **Pas d'index spatial (R-tree)** : les tuiles FLAIR-HUB sont sur une grille régulière. Un simple dictionnaire `(row, col)` → `Path` suffit.
- **Chargement lazy ou eager** : par défaut `preload=True` pour les performances.
- **Système de coordonnées** : EPSG :2154 (Lambert-93, mètres). Origine du pixel grid au nord-ouest.
- **Layout band-first** : tableaux toujours en `(bands, H, W)`, convention rasterio.

3.2 drone/ — État du drone et capteurs

Ce module modélise le drone (position, déplacement, bornes) et la caméra nadir qui convertit l'altitude en observation au sol.

Classe	Fichier	Rôle
`DroneConfig`	`drone.py`	Limites physiques immuables
`DroneState`	`drone.py`	Position courante (mutable)
`MoveResult`	`drone.py`	Résultat d'un déplacement
`Drone`	`drone.py`	Classe principale : reset, move
`CameraConfig`	`camera.py`	Paramètres du capteur
`CameraModel`	`camera.py`	Géométrie au sol, capture d'image
`TelemetryRecord`	`telemetry.py`	Snapshot gelé par pas de temps
`FlightLog`	`telemetry.py`	Journal de vol complet

Table 4 – Classes du module `drone/`

Décisions clés :

- **Mouvement instantané** : pas de simulation aérodynamique. Cohérent avec FlySearch.
- **Clampage aux bornes** : si un mouvement sortirait de la carte ou des limites d'altitude, la position est clampée silencieusement (`was_clipped=True`).
- **Copie d'état** : `Drone.state` renvoie une *copie* pour éviter les mutations accidentelles.

3.3 Modèle de caméra

La caméra pointe toujours au nadir (verticale). La couverture au sol dépend de l'altitude :

$$\text{empreinte} = 2 \times z \times \tan\left(\frac{\text{FOV}}{2}\right) \tag{1}$$

$$\text{GSD} = \frac{\text{empreinte}}{\text{image_size}} \tag{2}$$

Avec le FOV par défaut de 90°, l'empreinte vaut exactement $2z$ mètres.

Altitude (m)	Empreinte (m)	GSD (m/px)
10	20	0.04
50	100	0.20
100	200	0.40
200	400	0.80
500	1000	2.00

Table 5 – Couverture au sol selon l'altitude (FOV = 90°, image_size = 500 px)

3.4 core/ — Moteur de simulation

Ce module assemble la carte, le drone et la caméra en une boucle cohérente `reset()` → `step()` → `Observation`.

Classe	Fichier	Rôle
`ActionType`	`action.py`	Enum : MOVE, FOUND, STOP
`Action`	`action.py`	Commande de déplacement gelée
`EpisodeResult`	`observation.py`	Résultat d'épisode
`Observation`	`observation.py`	Sortie d'un pas : image, état, done
`SimulatorConfig`	`simulator.py`	Configuration globale
`FlairSimulator`	`simulator.py`	Classe principale du simulateur

Table 6 – Classes du module `core/`

Un épisode se termine lorsque :

1. L'agent appelle `Action.found()` (`success` selon l'évaluation).
2. L'agent appelle `Action.stop()` (`success=False`).
3. La limite de pas est atteinte (`success=False`).

3.5 server/ — API HTTP REST et SSE

Ce module expose le simulateur comme un service HTTP local via FastAPI.

Endpoint	Méthode	Description
`/reset`	POST	Démarrer un nouvel épisode
`/step`	POST	Envoyer une action, recevoir l'observation
`/state`	GET	État courant du drone
`/telemetry`	GET	Journal de vol complet
`/config`	GET	Configuration et bornes de la carte
`/events`	GET	Flux SSE d'observations temps réel

Table 7 – Endpoints du serveur

3.5.1 Server-Sent Events (SSE)

Le endpoint `GET /events` utilise `sse-starlette` pour diffuser chaque observation produite par `/reset` ou `/step` à tous les clients connectés en temps réel.

Architecture du système de push :

- Chaque viewer connecté reçoit une `asyncio.Queue(maxsize=64)`.
- La fonction `_broadcast()` sérialise l'`ObservationResponse` une seule fois et la pousse dans toutes les queues.
- **Backpressure** : si un consommateur est lent (queue pleine), l'événement le plus ancien est supprimé pour faire de la place.
- Un commentaire `keep-alive` est envoyé toutes les 30 secondes pour éviter les timeouts de proxy.
- Un commentaire initial `connected` est émis immédiatement pour flusher les en-têtes HTTP (le client voit le statut 200 sans attendre le premier événement).

Thread-safety : les endpoints `/reset` et `/step` sont `async def` et délèguent les appels lourds au simulateur via `asyncio.to_thread()`. Ainsi, `_broadcast()` s'exécute sur le thread de la boucle événementielle où `asyncio.Queue.put_nowait()` est sûr.

3.6 viewer/ — Visualisation desktop

Le viewer Pygame prend en charge trois modes de fonctionnement :

Mode	Flag	Description
local	`-mode local`	Crée un `FlairSimulator` local, vol direct. Mode par défaut.
observe	`-mode observe`	Se connecte au serveur via SSE (`GET /events`). Lecture seule.
fly	`-mode fly`	Se connecte au serveur, envoie `POST /step` sur les touches.

Table 8 – Modes du viewer

Le pattern **ViewerObservation** : les composants de rendu (HUD, minimap, image principale) consomment tous un `ViewerObservation` — un dataclass léger avec exactement les champs nécessaires au rendu. Deux méthodes factory le construisent depuis une `Observation` locale (`from_observation()`) ou un dict JSON serveur (`from_server_response()`), rendant le code de rendu agnostique de la source.

4 Données FLAIR-HUB

FlairSim utilise le jeu de données FLAIR-HUB [1] publié par l'IGN. Ce jeu de données couvre 78 départements français avec plusieurs modalités de télédétection.

4.1 Structure des données

```
D004-2021_AERIAL_RGBI/
  ROI_0001/
    D004_AERIAL_RGBI_ROI_0001_000-000.tif
    D004_AERIAL_RGBI_ROI_0001_000-001.tif
    ...
  ROI_0002/
    ...
```

Chaque patch est un GeoTIFF de 512×512 pixels en EPSG :2154 (Lambert-93) couvrant $102.4\,\mathrm{m} \times 102.4\,\mathrm{m}$ à une résolution de $0.2\,\mathrm{m/px}$. Les patches d'une ROI forment une grille contiguë que le simulateur assemble en une carte continue.

4.2 Modalités supportées

L'architecture du `MapManager` supporte 10 modalités FLAIR-HUB : AERIAL_RGBI, AERIAL_RGBI_LABELS, DEM, SPOT, SPOT_LABELS, SENTINEL1, SENTINEL2, SENTINEL2_LABELS, SENTINEL2_SUPERRES, SENTINEL2_SUPERRES_LABELS. Seule la modalité AERIAL_RGBI est utilisée pour le vol dans la version actuelle.

4.3 Données disponibles localement

Deux domaines sont présents sur la machine de développement :

- `D004-2021_AERIAL_RGBI/` — 3.8 Go, 38 ROI
- `D005-2018_AERIAL_RGBI/` — 2.2 Go, 28 ROI

5 Utilisation

5.1 Lancer le serveur

```
uv run python -m flairsim.server \
    --data-dir path/to/D004-2021_AERIAL_RGBI
```

Le serveur démarre sur `http://127.0.0.1:8000`. La documentation interactive (Swagger UI) est accessible à `http://127.0.0.1:8000/docs`.

Options CLI complètes :

Option	Défaut	Description
-host	127.0.0.1	Adresse de liaison
-port	8000	Port
-roi	auto	ROI à charger (la plus grande par défaut)
-max-steps	500	Nombre max de pas par épisode
-altitude	100	Altitude par défaut (mètres)
-image-size	500	Résolution caméra (pixels)
-fov	90	Champ de vision (degrés)
-no-preload		Ne pas précharger les tuiles
-v		Logs de débogage

Table 9 – Options de la ligne de commande du serveur

5.2 Piloter le drone depuis un programme externe

```python
import httpx
import base64

client = httpx.Client(base_url="http://127.0.0.1:8000")

# Demarrer un episode
obs = client.post("/reset").json()
print(f"Drone a ({obs['drone_state']['x']:.0f}, "
      f"{obs['drone_state']['y']:.0f})")

# Voler
while not obs["done"]:
    obs = client.post("/step", json={
        "dx": 10.0, "dy": 0.0, "dz": -2.0
    }).json()

    # Decoder l'image (PNG base64 -> bytes)
    image_bytes = base64.b64decode(obs["image_base64"])

# Resultat
if obs["done"]:
    print(obs["result"])
```

Listing 1 – Exemple Python avec httpx

Cela fonctionne depuis **n'importe quel langage** :

```bash
# Demarrer un episode
curl -X POST http://127.0.0.1:8000/reset

# Deplacer le drone
curl -X POST http://127.0.0.1:8000/step \
  -H "Content-Type: application/json" \
  -d '{"dx": 10, "dy": 5, "dz": 0}'

# Etat courant
curl http://127.0.0.1:8000/state

# Journal de vol
curl http://127.0.0.1:8000/telemetry
```

```
# Configuration
curl http://127.0.0.1:8000/config

# S'abonner au flux SSE
curl -N http://127.0.0.1:8000/events
```

Listing 2 – Exemples curl

5.3 Viewer interactif

5.3.1 Mode local (défaut)

Vol manuel au clavier avec simulateur local (pas de serveur nécessaire) :

```
uv run python -m flairsim.viewer \
    --data-dir path/to/D004-2021_AERIAL_RGBI
```

5.3.2 Mode observe

Connexion à un serveur et observation en temps réel de ce que fait un agent externe (lecture seule, pas de vol au clavier) :

```
# Terminal 1 : serveur
uv run python -m flairsim.server --data-dir path/to/data

# Terminal 2 : observer
uv run python -m flairsim.viewer \
    --mode observe --server-url http://127.0.0.1:8000
```

L'observateur se connecte via SSE (`GET /events`) et reçoit chaque observation poussée par le serveur.

5.3.3 Mode fly

Connexion à un serveur et pilotage du drone via les touches du clavier (envoie `POST /step` à chaque appui) :

```
uv run python -m flairsim.viewer \
    --mode fly --server-url http://127.0.0.1:8000
```

5.3.4 Contrôles clavier

Touche	Action
Z / ↑	Avancer (nord)
S / ↓	Reculer (sud)
Q / ←	Gauche (ouest)
D / →	Droite (est)
A	Descendre
E	Monter
+/−	Ajuster la taille du pas
Espace	Déclarer FOUND
R	Réinitialiser l'épisode
H	Afficher/cacher le HUD
M	Afficher/cacher la minimap
Échap	Quitter

Table 10 – Contrôles clavier du viewer

5.4 Format de la réponse d'observation

Chaque appel à `POST /reset` ou `POST /step` retourne un JSON avec la structure suivante :

```
{
  "step": 42,
  "done": false,
  "drone_state": {
    "x": 923456.2,
    "y": 6345123.8,
    "z": 100.0,
    "heading": 0.0,
    "step_count": 42,
    "total_distance": 523.4
  },
  "ground_footprint": 200.0,
  "ground_resolution": 0.4,
  "image_base64": "<PNG en base64>",
  "image_width": 500,
  "image_height": 500,
  "result": null,
  "metadata": {}
}
```

Listing 3 – ObservationResponse

Lorsque `done=true`, le champ `result` contient :

```
{
  "success": false,
  "reason": "Agent declared FOUND ...",
  "steps_taken": 42,
  "distance_travelled": 523.4
}
```

Listing 4 – EpisodeResultResponse

5.5 Types d'action

Le champ `action_type` dans `POST /step` accepte trois valeurs :

Type	Description
`move`	Déplacer le drone de `(dx, dy, dz)` mètres (défaut)
`found`	Déclarer la cible trouvée (termine l'épisode)
`stop`	Terminer volontairement l'épisode (`success=False`)

Table 11 – Types d'action pour `POST /step`

6 Choix de conception

6.1 Pourquoi pas un moteur 3D (UE5, Unity) ?

FlySearch utilise Unreal Engine 5, qui fournit des environnements photoréalistes en 3D mais nécessite une infrastructure lourde (GPU, installation du moteur de jeu, bridge C++). FlairSim échange délibérément la fidélité 3D contre :

1. **Imagerie réelle** : FLAIR-HUB fournit de vraies photos aériennes de la France.
2. **Simplicité** : pure Python, aucune dépendance à un moteur de jeu.
3. **Reproductibilité** : simulation déterministe.
4. **Diversité du dataset** : 78 départements, 19 classes d'occupation des sols, multiples capteurs.

Le compromis est que seules les vues nadir (plongeantes) sont supportées. C'est approprié pour de nombreuses tâches réelles de relevé par drone.

6.2 Pourquoi des déplacements relatifs ?

Conformément à FlySearch, les actions sont des `(dx, dy, dz)` en mètres. C'est :

- **Simple** : pas de planification de trajectoire.
- **VLM-friendly** : l'agent raisonne en "avancer de 10 m à l'est" plutôt qu'en coordonnées GPS absolues.
- **Compatible** : comparaison directe avec les résultats de FlySearch.

6.3 Pourquoi une API HTTP ?

Le simulateur et l'agent sont des préoccupations séparées. Une API HTTP :

- **Découple** le simulateur de l'implémentation de l'agent.
- **Reflète les systèmes réels** où les composants d'un drone communiquent via des API.
- **Permet le développement parallèle** : l'équipe simulateur et l'équipe VLM travaillent indépendamment.
- **Documentation gratuite** via le Swagger UI de FastAPI.

6.4 Pourquoi des dataclasses gelés ?

Les objets de configuration (`DroneConfig`, `CameraConfig`, `TelemetryRecord`) sont tous décorés avec `@dataclass(frozen=True, slots=True)`. Cela fournit :

- **Thread safety** : les objets immuables sont partageables.
- **Hashabilité** : utilisable comme clé de dictionnaire.
- **Prévention des bugs** : mutation accidentelle détectée à l'exécution.
- **Efficacité mémoire** : `slots=True` élimine le surcoût de `__dict__`.

7 Flux de données

Le diagramme suivant illustre le flux complet, incluant le canal SSE pour les observateurs :

```
Agent externe (VLM, script)          Viewer observateur (SSE)
       |                                      ^
       |  HTTP POST /step                     | GET /events
       v                                      |
  flairsim.server (FastAPI) -----------> _broadcast()
       |                                 (push dans toutes
       v                                  les queues)
  FlairSimulator.step(action)
       |
       +---> Drone.move(dx, dy, dz)
       |          |
       |          v
       |     DroneState (position clampee)
       |
       +---> CameraModel.capture(map_manager, x, y, z)
       |          |
       |          v
       |     MapManager.get_region()
       |          |
       |          v
       |     np.ndarray (bands, H, W)
       |
       +---> FlightLog.append(TelemetryRecord)
       |
       v
  Observation (image, drone_state, done, result)
       |
       +---> JSON response --> Agent externe
       |
       +---> ObservationResponse.model_dump_json()
                  |
                  v
             SSE "observation" event --> Viewers observateurs
```

8 Tests

La suite de tests comprend **268 tests** organisés en quatre catégories :

Fichier	Nb	Portée
`test_action.py`	13	Action, ActionType
`test_camera.py`	15	CameraConfig, CameraModel
`test_drone.py`	39	DroneConfig, DroneState, Drone
`test_map_manager.py`	44	MapManager (intégration, GeoTIFFs synthétiques)
`test_modality.py`	11	Modality, ModalitySpec
`test_observation.py`	14	Observation, EpisodeResult
`test_server.py`	42	Endpoints HTTP + SSE
`test_simulator.py`	47	FlairSimulator (intégration)
`test_telemetry.py`	16	TelemetryRecord, FlightLog
`test_tile_loader.py`	16	TileInfo, TileData
Total	**268**	

Table 12 – Répartition des tests

- **Tests unitaires** (171) : classes testées en isolation avec des tableaux numpy et des mocks.
- **Tests d'intégration** (91) : grilles GeoTIFF synthétiques créées avec `rasterio`.
- **Tests serveur** (36) : `TestClient` de FastAPI avec données synthétiques.
- **Tests SSE** (6) : serveur `uvicorn.Server` réel dans un thread, connexion via `httpx.AsyncClient` et `asyncio.wait()` pour souscription et déclenchement concurrents.

Lancer les tests :

```
uv run pytest                # tous les 268 tests (~5s)
uv run pytest -v             # verbeux
uv run pytest -x             # arreter au premier echec
uv run pytest -k "drone"     # tests lies au drone
uv run pytest -k "server"    # tests serveur + SSE
```

9 Structure du projet

```
simulateur/
  flairsim/                    # Package principal
    __init__.py                # API publique (17 exports)
    map/                       # Chargement GeoTIFF, requetes spatiales
      modality.py              # Enum Modality
      tile_loader.py           # TileInfo, TileData, read_tile()
      map_manager.py           # MapManager, MapBounds
    drone/                     # Etat du drone, physique, camera
      drone.py                 # Drone, DroneConfig, DroneState
      camera.py                # CameraModel, CameraConfig
      telemetry.py             # FlightLog, TelemetryRecord
    core/                      # Moteur de simulation
      action.py                # Action, ActionType
      observation.py           # Observation, EpisodeResult
      simulator.py             # FlairSimulator, SimulatorConfig
    server/                    # API HTTP REST + SSE
      __main__.py              # python -m flairsim.server
      app.py                   # Factory FastAPI, routes, broadcast SSE
      cli.py                   # Parsing CLI
    viewer/                    # Visualisation desktop (pygame, 3 modes)
      __main__.py              # python -m flairsim.viewer
      viewer.py                # FlairViewer, controles clavier, modes
      remote.py                # ViewerObservation (adapteur
         local/serveur)
      minimap.py               # Overlay minimap
      hud.py                   # Overlay HUD
  tests/                       # 268 tests
  docs/                        # Documentation technique
    api.md                     # Reference API complete
    architecture.md            # Architecture et rationale
    contributing.md            # Guide de developpement
    guide.tex                  # Ce document
  pyproject.toml               # Config package (uv/hatchling)
  uv.lock                     # Lockfile (reproductibilite)
  .python-version              # Version Python (3.11)
  README.md                    # Vue d'ensemble du projet
```

10 Utilisation programmatique (in-process)

FlairSim peut aussi être utilisé directement en tant que bibliothèque Python, sans passer par le serveur HTTP :

```python
from flairsim import FlairSimulator, Action

sim = FlairSimulator(data_dir="path/to/D004-2021_AERIAL_RGBI")
obs = sim.reset()

while not obs.done:
    # Logique de l'agent
    action = Action.move(dx=10.0, dy=0.0, dz=0.0)
    obs = sim.step(action)

    # Acces a l'image RGB (H, W, 3)
    rgb = obs.image_rgb()

    # Acces a l'etat du drone
    print(f"Position: ({obs.drone_state.x:.0f}, "
          f"{obs.drone_state.y:.0f}, "
          f"{obs.drone_state.z:.0f})")

# Resultat de l'episode
print(f"Succes: {obs.result.success}")
print(f"Raison: {obs.result.reason}")
print(f"Distance: {obs.result.distance_travelled:.1f} m")
```

Listing 5 – Utilisation in-process

11 Qualité du code

11.1 Linting et formatage

Le projet utilise `ruff` pour le linting et le formatage :

```bash
# Verifier
uv run ruff check flairsim/ tests/

# Corriger automatiquement
uv run ruff check --fix flairsim/ tests/

# Formater
uv run ruff format flairsim/ tests/
```

Règles activées : E/W (pycodestyle), F (pyflakes), I (isort), N (pep8-naming), D (pydocstyle), UP (pyupgrade).

11.2 Conventions

- **Longueur de ligne** : 88 caractères (compatible Black)
- **Version Python** : 3.11 (épinglée)
- **Docstrings** : style NumPy avec `Parameters`, `Returns`, `Raises`
- **Type hints** : toutes les API publiques sont typées

— **Dataclasses gelés** : `@dataclass(frozen=True, slots=True)` pour les configurations et objets-valeurs
— **Logging** : `logging.getLogger(__name__)` dans chaque module, jamais de `print()`
— **Build backend** : hatchling (pas setuptools)

12 Références

Références

[1] Music, A. et al. (2025). *FLAIR-HUB : A large-scale multimodal dataset for land-cover mapping.* arXiv :2506.07080.

[2] Majumdar, A. et al. (2025). *FlySearch : A benchmark for evaluating VLM exploration capabilities with a UAV.* NeurIPS 2025. arXiv :2506.02896.